

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder

plt.style.use('default')
```

▼ Problem 1 to 2:

Dataset link: <https://rb.gy/gsmddu>

Add a label to every axis and add a proper title for the charts (For every subplot, it is applicable). Also add proper labels if there are multiple representations. Then, you can customize it as your wish.

▼ Problem-1: Make a subplots which have 2 plots.

- For the first chart, draw a scatter plot "*Monitored Cap.(MW)*" vs "*Total Cap. Under Maintenance (MW)*" of top 5 most frequent power stations. Then draw the lines which indicate the average values of these two columns. Change the colors according to the names of the Power Stations.
- For the second chart, draw a scatter plot "*Monitored Cap. (MW)*" vs "*Actual(MU)*" of the top 5 most frequent power stations. Also draw the lines which indicates the average values of these two columns. Change the colors according to the names of the Power Stations.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("/content/PowerGeneration - PowerGeneration.csv")

top5 = df['Power Station'].value_counts().head(5).index

top_df = df[df['Power Station'].isin(top5)]

avg_monitored_1 = top_df['Monitored Cap.(MW)'].mean()
avg_maintenance = top_df['Total Cap. Under Maintenace (MW)'].mean()

avg_monitored_2 = top_df['Monitored Cap.(MW)'].mean()
avg_actual = top_df['Actual(MU)'].mean()

fig, ax = plt.subplots(1,2,figsize=(18,7))

for station in top5:

    station_df = top_df[top_df['Power Station'] == station]

    ax[0].scatter(station_df['Monitored Cap.(MW)'],
                  station_df['Total Cap. Under Maintenance (MW)'],
                  label=station,
                  alpha=0.7,
                  s=80)

    ax[0].axvline(avg_monitored_1,color='red',linestyle='--',linewidth=2,label='Avg Monitored Cap.')
    ax[0].axhline(avg_maintenance,color='blue',linestyle='--',linewidth=2,label='Avg Maintenance Cap.')

    ax[0].set_xlabel('Monitored Cap. (MW)')
    ax[0].set_ylabel('Total Cap. Under Maintenance (MW)')
    ax[0].set_title('Monitored Capacity vs Maintenance Capacity')
    ax[0].grid(True)
    ax[0].legend()

for station in top5:

    station_df = top_df[top_df['Power Station'] == station]

    ax[1].scatter(station_df['Monitored Cap.(MW)'],
                  station_df['Actual(MU)'],
                  label=station,
                  alpha=0.7,
                  s=80)

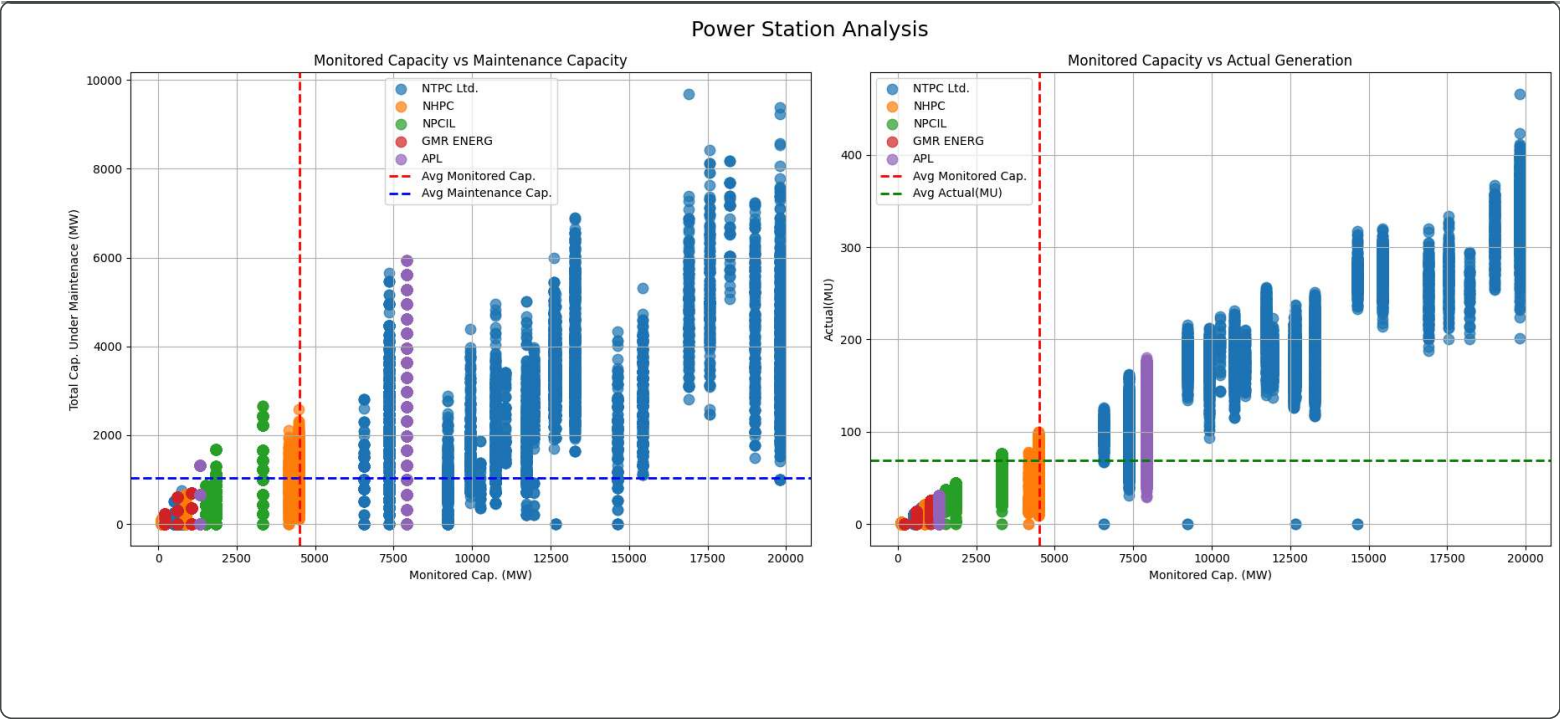
    ax[1].axvline(avg_monitored_2,color='red',linestyle='--',linewidth=2,label='Avg Monitored Cap.')
    ax[1].axhline(avg_actual,color='green',linestyle='--',linewidth=2,label='Avg Actual(MU)')

    ax[1].set_xlabel('Monitored Cap. (MW)')
    ax[1].set_ylabel('Actual(MU)')
    ax[1].set_title('Monitored Capacity vs Actual Generation')
    ax[1].grid(True)
    ax[1].legend()

plt.suptitle('Power Station Analysis',fontsize=18)

plt.tight_layout()

plt.show()
```



df											
	Dates	Power Station	Monitored Cap.(MW)	Total Cap. Under Maintenance (MW)	Planned Maintenance (MW)	Forced Maintenance(MW)	Other Reasons (MW)	Programme or Expected(MU)	Actual(MU)	Excess(+) / Shortfall (-)	Deviation
0	2017-09-01	Delhi	2235.4	135.00	0.00	135.0	0	13.0	18.0	5.00	0.0
1	2017-09-01	Haryana	2720.0	2470.00	0.00	2470.0	0	28.0	7.0	-21.80	0.0
2	2017-09-01	Himachal Pradesh	3378.0	379.00	0.00	231.0	0	40.0	46.0	5.63	0.0
3	2017-09-01	Jammu and Kashmir	1285.0	150.00	0.00	0.0	0	14.0	23.0	9.43	0.0
4	2017-09-01	Punjab	3826.3	2697.65	77.65	2620.0	0	39.0	17.0	-21.69	0.0
...	...	...	...	...	...	...	...	...	...	...	...
135510	2019-10-20	LPGCL	1980.0	660.00	0.00	660.0	0	30.0	20.0	-9.95	0.0
135511	2019-10-20	L&T	0.0	0.00	0.00	0.0	0	0.0	0.0	0.00	0.0
135512	2019-10-20	MPCL	86.0	0.00	0.00	0.0	0	0.0	0.0	-0.15	0.0
135513	2019-10-20	MUNPL	660.0	660.00	0.00	660.0	0	8.0	0.0	-8.90	0.0
135514	2019-10-20	NHPC	4484.2	797.00	370.00	0.0	0	NaN	NaN	NaN	NaN

▼ **Problem-2:** Draw a 3D Scatter plot between "*Monitored Cap.(MW)*", "*Total Cap. Under Maintanace (MW)*" and "*Forced Maintanence(MW)*"

```
# code here
fig = plt.figure(figsize=(12,8))

ax = fig.add_subplot(111,projection='3d')

scatter = ax.scatter(df['Monitored Cap.(MW)'],
                    df['Total Cap. Under Maintenance (MW)'],
                    df['Forced Maintenance(MW)'],
                    c=df['Forced Maintenance(MW)'],
                    cmap='viridis',
                    alpha=0.7,
                    s=40)

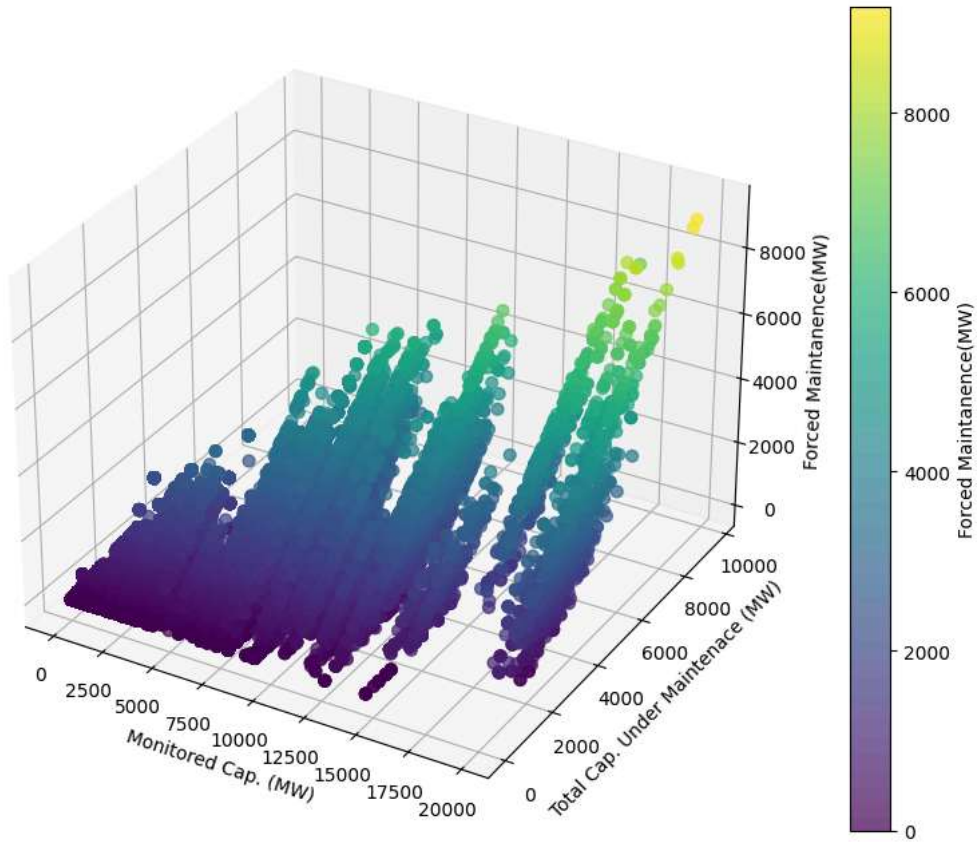
ax.set_xlabel('Monitored Cap. (MW)')
ax.set_ylabel('Total Cap. Under Maintenance (MW)')
ax.set_zlabel('Forced Maintenance(MW)')

ax.set_title('3D Scatter Plot of Power Generation Data')

fig.colorbar(scatter,label='Forced Maintenance(MW)')

plt.show()
```

3D Scatter Plot of Power Generation Data



▼ **Problem-3:** Make a 3D *Surface* plot of this below mathematical equation.

$$z = |x| - |y|$$

```
# code here
x = np.linspace(-10,10,100)
y = np.linspace(-10,10,100)

X,Y = np.meshgrid(x,y)

Z = np.abs(X) - np.abs(Y)

fig = plt.figure(figsize=(12,8))

ax = fig.add_subplot(111,projection='3d')

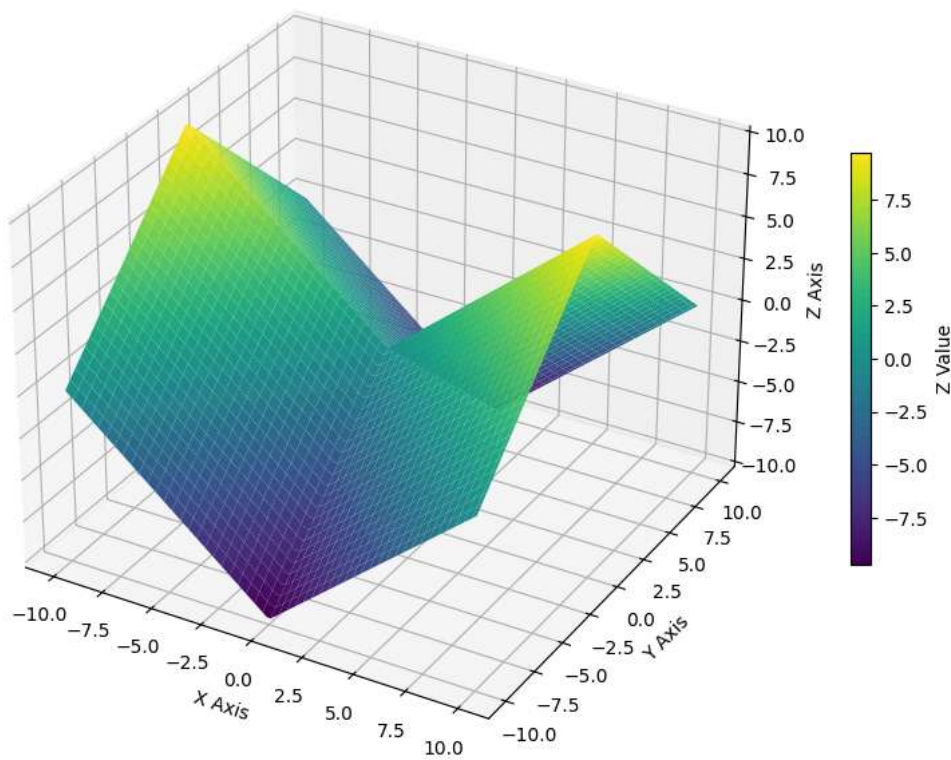
surface = ax.plot_surface(X,Y,Z,cmap='viridis')

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')

ax.set_title('3D Surface Plot of z = |x| - |y|')

fig.colorbar(surface,shrink=0.5,label='Z Value')

plt.show()
```

3D Surface Plot of $z = |x| - |y|$ 

✓ **Problem-4:** Draw the 3D *Contour* plot of this below equation:

$$z = |x| - |y|$$

```
# code here
x = np.linspace(-10,10,100)
y = np.linspace(-10,10,100)

X,Y = np.meshgrid(x,y)

Z = np.abs(X) - np.abs(Y)

fig = plt.figure(figsize=(12,8))

ax = fig.add_subplot(111,projection='3d')

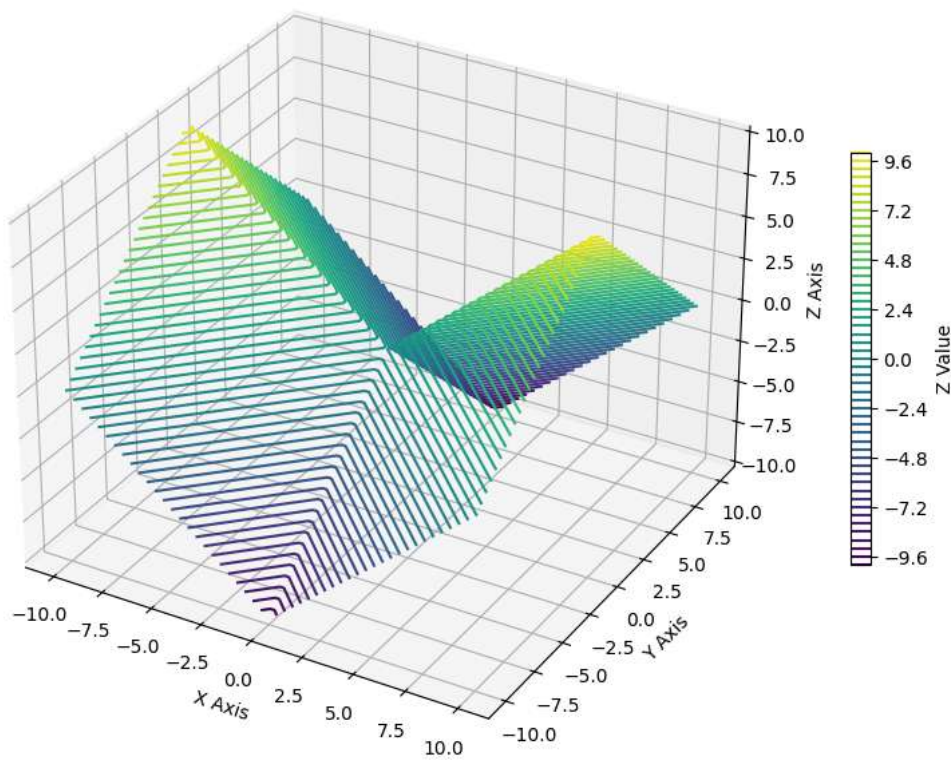
contour = ax.contour3D(X,Y,Z,50,cmap='viridis')

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')

ax.set_title('3D Contour Plot of  $z = |x| - |y|$ ')

fig.colorbar(contour,shrink=0.5,label='Z Value')

plt.show()
```

3D Contour Plot of $z = |x| - |y|$ 

▼ **Problem-5:** Draw a second type of Countour plot of the below equation:

$$z = |x| - |y|$$

```
# code here
x = np.linspace(-10,10,100)
y = np.linspace(-10,10,100)

X,Y = np.meshgrid(x,y)

Z = np.abs(X) - np.abs(Y)

fig = plt.figure(figsize=(10,8))

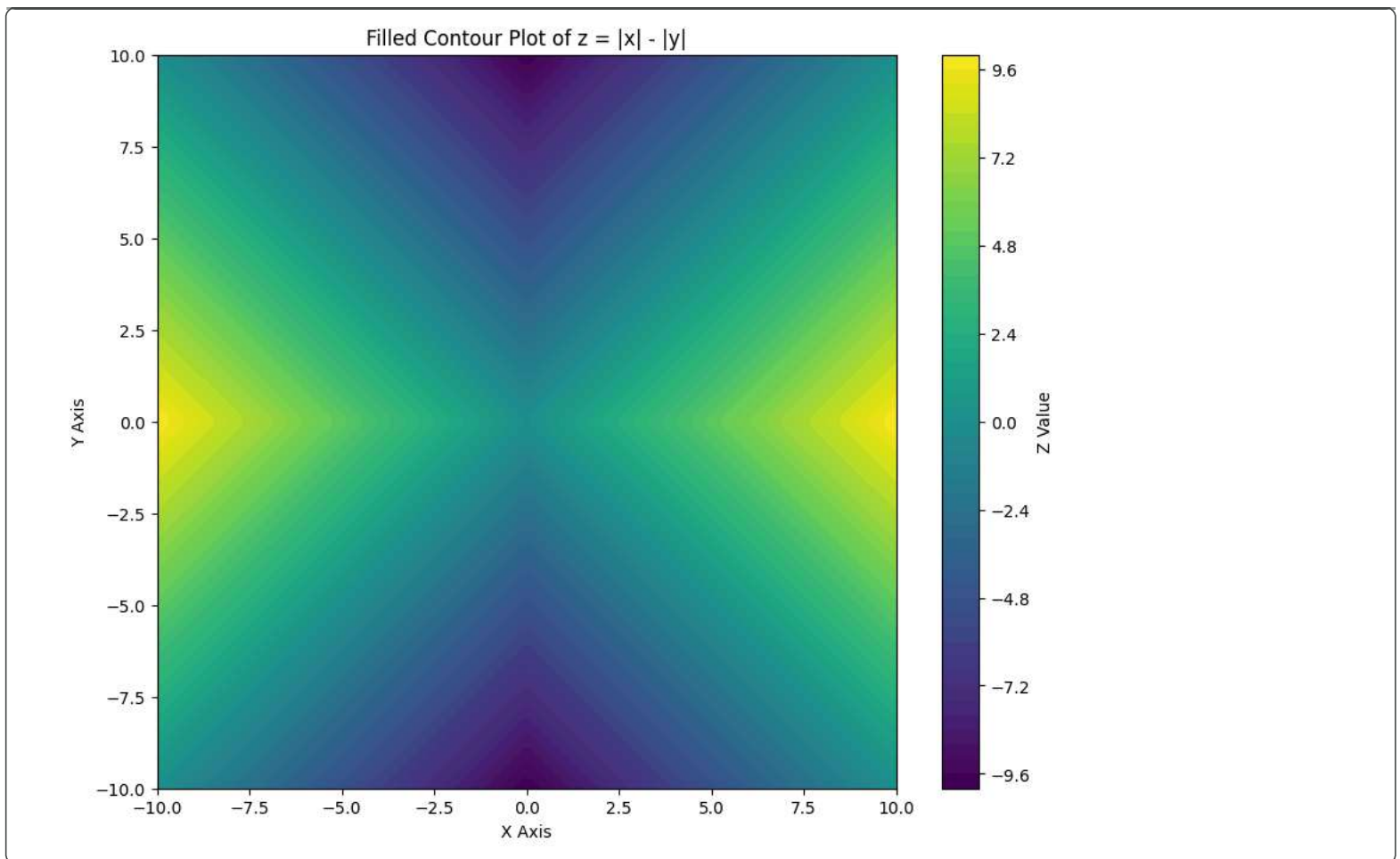
contour = plt.contourf(X,Y,Z,50,cmap='viridis')

plt.xlabel('X Axis')
plt.ylabel('Y Axis')

plt.title('Filled Contour Plot of  $z = |x| - |y|$ ')

plt.colorbar(contour,label='Z Value')

plt.show()
```

Problem 6-7

Data Set Link - https://docs.google.com/spreadsheets/d/17tUL2yC7MGvo7txuuhLtAl-b6_C4jcOt7FLExqRm-ul/edit?usp=share_link

Description of Dataset:

- Date: It gives the date of which stocks details are given.
- Symbol: Name of stock
- Open: It gives the opening price of stock on that date.
- High: It gives the highest price to which the stock ascended on that day.
- Low: It gives the highest price to which the stock plummeted on that day.
- Close: It gives the closing price of stock on that date.
- Volume: It gives the amount of stock traded on that date.
- VWAP: The volume-weighted average price (VWAP) is a statistic used by traders to determine what the average price is based on both price and volume.
- Turnover:

Problem-6 Use Pandas plot functions

- Line plot of closing value of top 5 Stocks in Year 2020.
- Take top 5 stocks based on total turnover in Year 2020

```
# code here
df = pd.read_csv('/content/nifty-50 - nifty-50.csv')

df.head()

df['Date'] = pd.to_datetime(df['Date'])

df_2020 = df[df['Date'].dt.year == 2020]

top5 = df_2020.groupby('Symbol')['Turnover'].sum().sort_values(ascending=False).head(5).index

top_df = df_2020[df_2020['Symbol'].isin(top5)]

pivot_df = top_df.pivot_table(index='Date',
                               columns='Symbol',
                               values='Close')

pivot_df.plot(figsize=(15,7),
               linewidth=2)

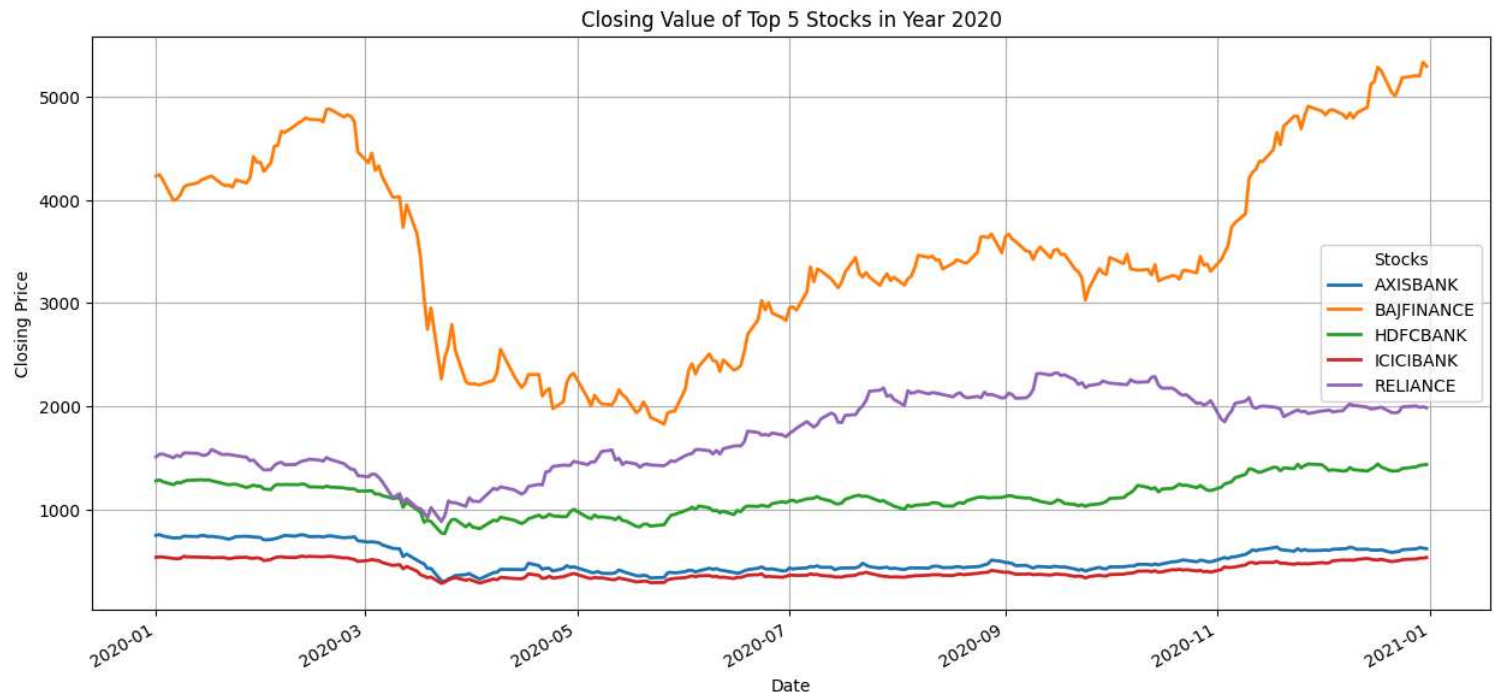
plt.xlabel('Date')
plt.ylabel('Closing Price')

plt.title('Closing Value of Top 5 Stocks in Year 2020')
```

```
plt.grid(True)

plt.legend(title='Stocks')

plt.show()
```



▼ **Problem-7** Scatter plot Close price vs Volume for TOP-5 Stocks in year 2021

- Color on Symbol Column

```
# code here
df['Date'] = pd.to_datetime(df['Date'])

df_2021 = df[df['Date'].dt.year == 2021]

top5 = df_2021.groupby('Symbol')['Turnover'].sum().sort_values(ascending=False).head(5).index

top_df = df_2021[df_2021['Symbol'].isin(top5)]

plt.figure(figsize=(15,7))

for stock in top5:

    stock_df = top_df[top_df['Symbol'] == stock]

    plt.scatter(stock_df['Close'],
                stock_df['Volume'],
                label=stock,
                alpha=0.7,
                s=50)

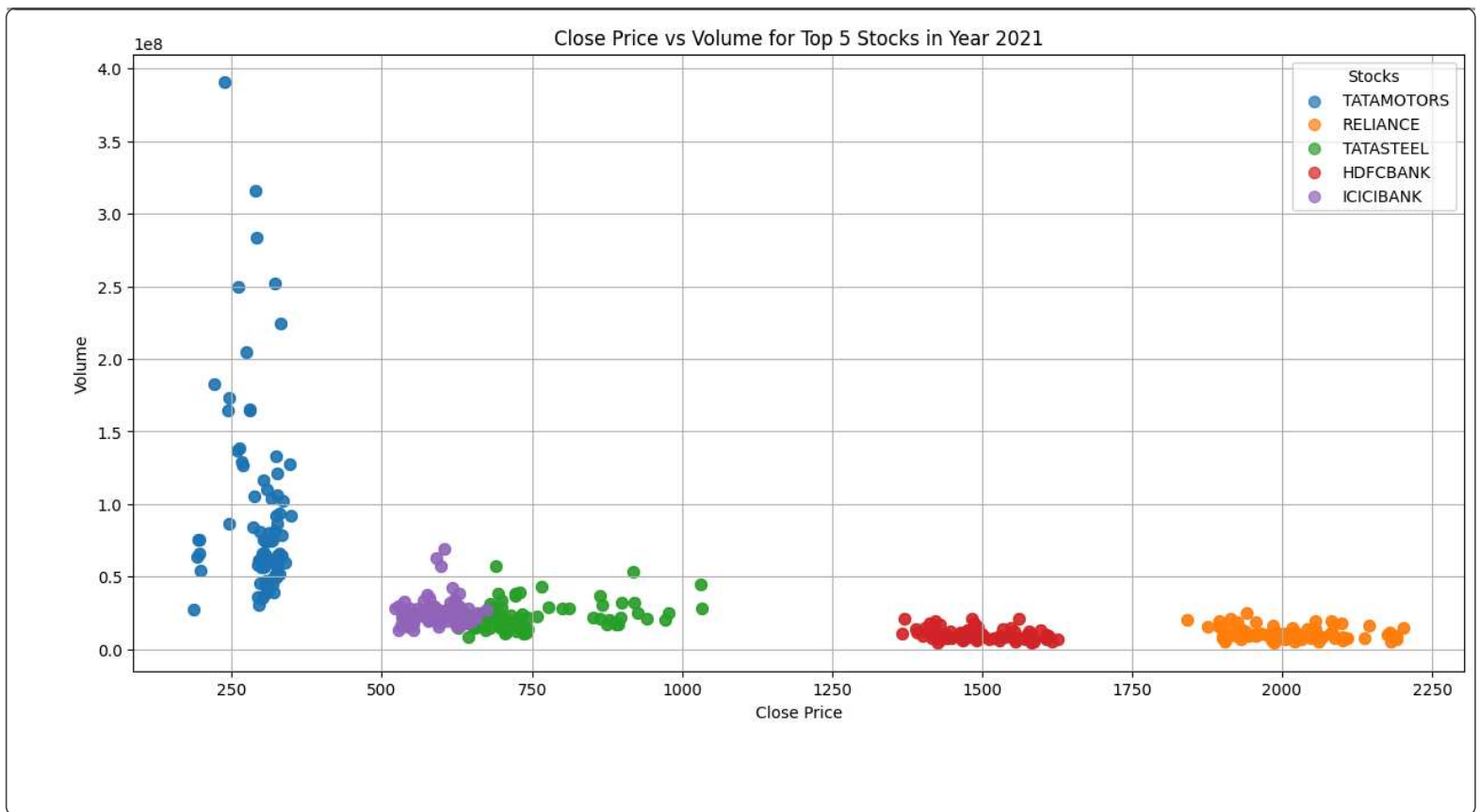
plt.xlabel('Close Price')
plt.ylabel('Volume')

plt.title('Close Price vs Volume for Top 5 Stocks in Year 2021')

plt.grid(True)

plt.legend(title='Stocks')

plt.show()
```



▼ **Problem-8** Create a 3-D Scatter Plot using (time,x,y) on below synthetic data. and give color gradient on (z)

```
# Create a 3D dataset
time = np.linspace(0, 10, 100)
x = np.sin(time)
y = np.cos(time)
z = time

# Create a DataFrame from the dataset
data = pd.DataFrame({'time': time, 'x': x, 'y': y, 'z': z})
```

```
# code here
time = np.linspace(0,10,100)

x = np.sin(time)
y = np.cos(time)
z = time

data = pd.DataFrame({'time':time,
                    'x':x,
                    'y':y,
                    'z':z})

fig = plt.figure(figsize=(12,8))

ax = fig.add_subplot(111,projection='3d')

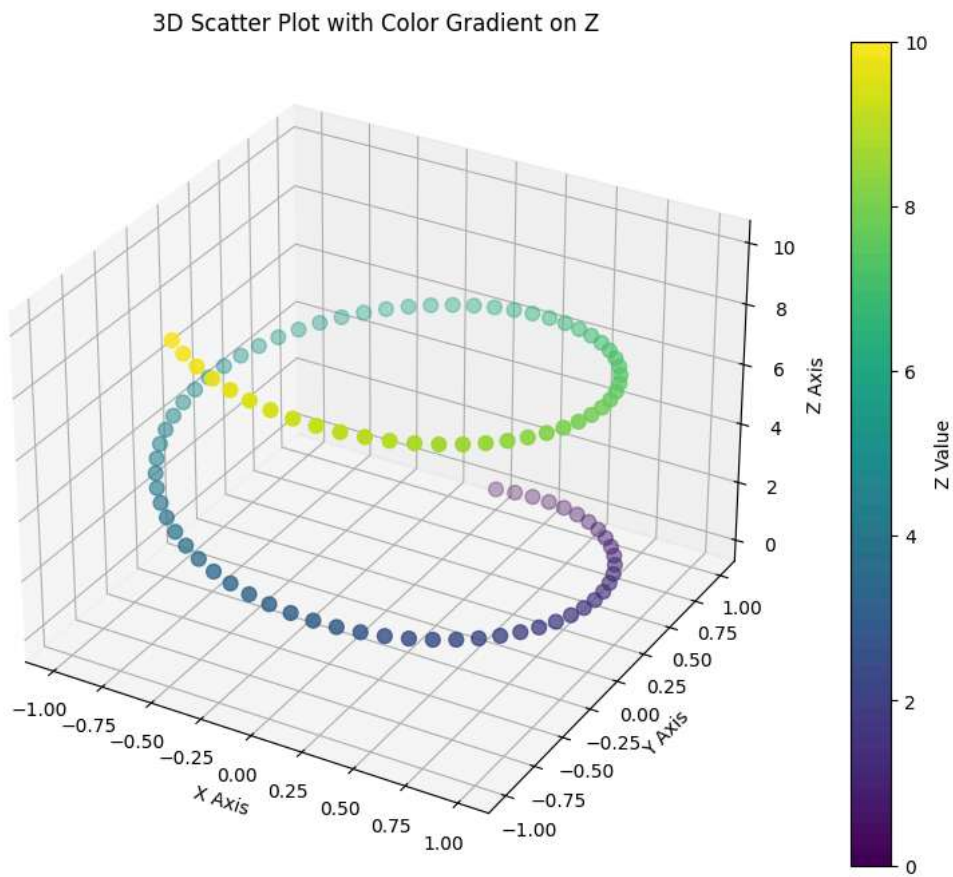
scatter = ax.scatter(data['x'],
                    data['y'],
                    data['z'],
                    c=data['z'],
                    cmap='viridis',
                    s=60)

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')

ax.set_title('3D Scatter Plot with Color Gradient on Z')

fig.colorbar(scatter,label='Z Value')

plt.show()
```

▼ **Problem 9:** Create a surface plot and the 2 types of the contour plots of the below equation.

$$z = \sin(\sqrt{x^2 + y^2})$$

```
# code here
x = np.linspace(-5,5,100)
y = np.linspace(-5,5,100)

X,Y = np.meshgrid(x,y)

Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure(figsize=(12,8))

ax = fig.add_subplot(131,projection='3d')

surface = ax.plot_surface(X,Y,Z,cmap='viridis')

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')

ax.set_title('3D Surface Plot')

ax2 = fig.add_subplot(132)

contour1 = ax2.contour(X,Y,Z,20,cmap='plasma')

ax2.set_xlabel('X Axis')
ax2.set_ylabel('Y Axis')

ax2.set_title('Contour Plot')

fig.colorbar(contour1,ax=ax2)

ax3 = fig.add_subplot(133)

contour2 = ax3.contourf(X,Y,Z,20,cmap='coolwarm')

ax3.set_xlabel('X Axis')
ax3.set_ylabel('Y Axis')

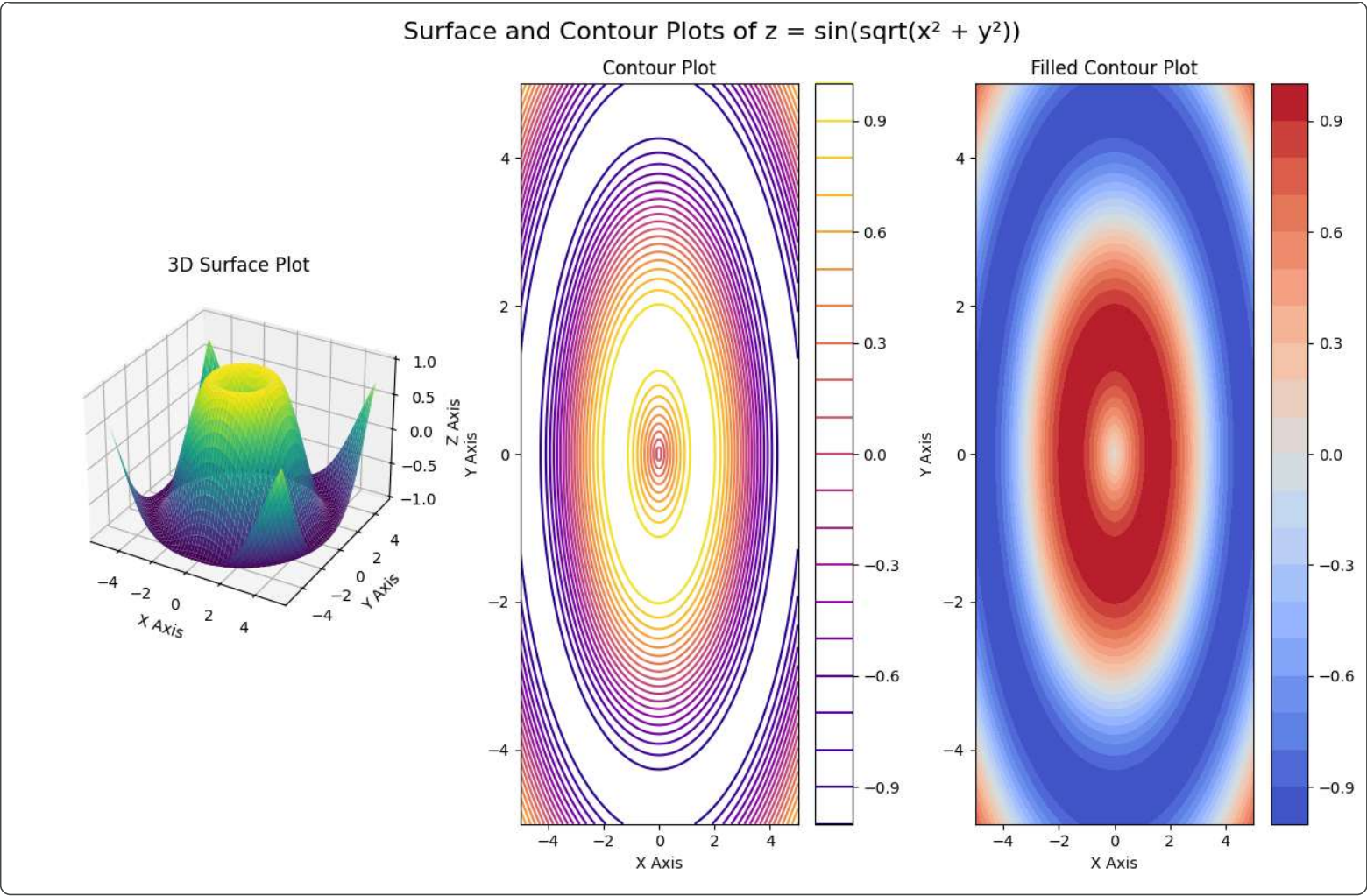
ax3.set_title('Filled Contour Plot')

fig.colorbar(contour2,ax=ax3)

plt.suptitle('Surface and Contour Plots of z = sin(sqrt(x^2 + y^2))',fontsize=16)

plt.tight_layout()

plt.show()
```



⌵ **Problem 10:** Create a surface plot and the 2 types of the contour plots of the below equation.

$$z = \tan(\log_2(x^2 + y^2))$$

```
# code here
```